

```

package com.tuapp.skin;

import android.hardware.usb.UsbDeviceConnection;
import android.util.Log;

/**
 * Control de las 8 luces del analizador facial MJ-008 (cámara UVC 0edc:3080).
 *
 * Protocolo extraído por ingeniería inversa de interoperabilidad del firmware/ap
 * original. Uso propio sobre hardware propio.
 *
 * Mecanismo: escritura a una Extension Unit (XU) de UVC mediante dos control
 * transfers consecutivos sobre la interfaz de VideoControl (interfaz 0):
 *
 * 1) Paquete de COMANDO (wValue = 0x0A00) - 8 bytes:
 *
 *   [0] (opcode >> 8) & 0xFF      = 0x00
 *   [1] opcode          & 0xFF      = 0x82
 *   [2] addr            & 0xFF      = 0x16
 *   [3] (addr >> 8)     & 0xFF      = 0xD8
 *   [4] len             & 0xFF      = 0x04
 *   [5] (len >> 8)      & 0xFF      = 0x00
 *   [6] (addr >> 16)    & 0xFF      = 0x00
 *   [7] 0x00
 *
 * 2) Paquete de DATOS (wValue = 0x0B00) - 8 bytes (payload de 4 + relleno):
 *
 *   [0] 0x00
 *   [1] 0x78 (constante del protocolo)
 *   [2] canal de luz (0x10..0x17)
 *   [3] 0xFF = encender / 0x00 = apagar
 *   [4..7] 0x00
 *
 * En ambos: bmRequestType = 0x21, bRequest = 0x01 (SET_CUR),
 *           wIndex = 0x0400 (Unit ID 4 << 8 | interfaz 0), wLength = 8.
 * El firmware original espera ~1 ms entre ambos paquetes.
 *
 * IMPORTANTE: la interfaz 0 debe estar reclamada (claimInterface) por la misma
 * UsbDeviceConnection antes de llamar a estos métodos.
 */
public class MaokinLightController {

    private static final String TAG = "MaokinLight";

    // ---- Canales de luz (byte [2] del payload) ----
    public static final int LIGHT_WHITE = 0x10; // luz blanca -> white.jp

```

```

public static final int LIGHT_XPL      = 0x11; // polarizada neg. -> negative
public static final int LIGHT_PPL      = 0x12; // polarizada pos. -> positive
public static final int LIGHT_WOODS    = 0x13; // lámpara de Wood -> wsg.jpg
public static final int LIGHT_UV       = 0x14; // ultravioleta    -> uv.jpg
public static final int LIGHT_BLUE     = 0x15; // luz azul        -> blue.jpg
public static final int LIGHT_BROWN   = 0x16; // luz marrón      -> orange.j
public static final int LIGHT_RED      = 0x17; // luz roja        -> red.jpg

/** Secuencia exacta que usa la app original, en orden. */
public static final int[] CAPTURE_SEQUENCE = {
    LIGHT_WHITE, LIGHT_XPL, LIGHT_PPL, LIGHT_WOODS,
    LIGHT_UV,    LIGHT_BLUE, LIGHT_BROWN, LIGHT_RED
};

/** Nombre de archivo que la app original asocia a cada canal. */
public static String fileNameFor(int channel) {
    switch (channel) {
        case LIGHT_WHITE: return "white.jpg";
        case LIGHT_XPL:   return "negative.jpg";
        case LIGHT_PPL:   return "positive.jpg";
        case LIGHT_WOODS: return "wsg.jpg";
        case LIGHT_UV:    return "uv.jpg";
        case LIGHT_BLUE:  return "blue.jpg";
        case LIGHT_BROWN: return "orange.jpg";
        case LIGHT_RED:   return "red.jpg";
        default:          return "unknown.jpg";
    }
}

// ---- Constantes del protocolo XU ----
private static final int REQ_TYPE_OUT = 0x21; // Host->Device | Class | Inter
private static final int SET_CUR      = 0x01;
private static final int WVAL_CMD     = 0x0A00;
private static final int WVAL_DATA    = 0x0B00;
private static final int WINDEX_XU    = 0x0400; // Unit ID 4, interfaz 0
private static final int PKT_LEN      = 8;
private static final int TIMEOUT_MS   = 1000;

private static final int OPCODE       = 0x0082;
private static final int LIGHT_ADDR   = 0x00D816;
private static final int PAYLOAD_LEN  = 4;

private final UsbDeviceConnection conn;

public MaokinLightController(UsbDeviceConnection conn) {
    this.conn = conn;
}

```

```

/** Enciende un canal de luz. */
public boolean turnOn(int channel) {
    return write(channel, 0xFF);
}

/** Apaga todas las luces (equivale al code 1011 de la app original). */
public boolean turnOff() {
    return write(LIGHT_WOODS, 0x00);
}

/**
 * Escribe directamente un canal y valor.
 * @param channel 0x10..0x17
 * @param value 0xFF encender, 0x00 apagar
 */
public boolean write(int channel, int value) {
    byte[] payload = new byte[] {
        0x00,
        (byte) 0x78,
        (byte) channel,
        (byte) value
    };
    return xuWrite(OPCODE, LIGHT_ADDR, payload);
}

/**
 * Implementación genérica de la escritura XU: paquete de comando seguido
 * de los paquetes de datos en bloques de 8 bytes.
 */
public boolean xuWrite(int opcode, int addr, byte[] payload) {
    if (conn == null) {
        Log.e(TAG, "UsbDeviceConnection nula");
        return false;
    }

    int len = payload.length;

    byte[] cmd = new byte[PKT_LEN];
    cmd[0] = (byte) ((opcode >> 8) & 0xFF);
    cmd[1] = (byte) (opcode & 0xFF);
    cmd[2] = (byte) (addr & 0xFF);
    cmd[3] = (byte) ((addr >> 8) & 0xFF);
    cmd[4] = (byte) (len & 0xFF);
    cmd[5] = (byte) ((len >> 8) & 0xFF);
    cmd[6] = (byte) ((addr >> 16) & 0xFF);
    cmd[7] = 0x00;

```

```

int r = conn.controlTransfer(REQ_TYPE_OUT, SET_CUR, WVAL_CMD,
                             WINDEX_XU, cmd, PKT_LEN, TIMEOUT_MS);

if (r < 0) {
    Log.e(TAG, "Fallo en paquete de comando: " + r);
    return false;
}

// Fase de datos: bloques de 8 bytes, el último relleno con ceros.
int blocks = (len + PKT_LEN - 1) / PKT_LEN;
for (int b = 0; b < blocks; b++) {
    byte[] chunk = new byte[PKT_LEN];
    int copy = Math.min(PKT_LEN, len - b * PKT_LEN);
    System.arraycopy(payload, b * PKT_LEN, chunk, 0, copy);

    r = conn.controlTransfer(REQ_TYPE_OUT, SET_CUR, WVAL_DATA,
                             WINDEX_XU, chunk, PKT_LEN, TIMEOUT_MS);

    if (r < 0) {
        Log.e(TAG, "Fallo en paquete de datos " + b + ": " + r);
        return false;
    }

    try { Thread.sleep(1); } catch (InterruptedException ignored) {}
}

return true;
}

// -----
// Tiempos que usa la app original entre pasos de la secuencia (ms).
// Encender luz -> esperar -> disparar -> esperar -> siguiente luz.
// -----
public static final long DELAY_FIRST_SHOT = 1500;
public static final long DELAY_BETWEEN    = 3600;
public static final long DELAY_AFTER_SHOT = 2000;
}

```